Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)
Department of CIS, Room F-1030-I
Office hours: Mon 2pm-5pm (in-person and Zoom)
Office phone: 212-220-1490
Borough of Manhattan Community College/CUNY

CSC331 Fall 2025
Data Structures

Week 5

Exam 1
Queues
Queue class definitions
Graph definitions and notations
Binary tree definitions and notations
Heap – priority queue

Professor Kevin Byron
Department of CIS
Borough of Manhattan Community College/CUNY

Reminders

- No class Wed 10-1-2025, Thu 10-2-2025
- Exam 1
  - Section 501 Tue 9-30-2025 – Zoom 630pm-730pm
  - Section 172 Wed 10-8-2025 – In person room M-1001 630pm-730pm
    - Closed book, closed notes, no devices
  - Section 500 Wed 10-8-2025 – In person room N-453 1010am-1110am
    - Closed book, closed notes, no devices
- Program 2 due date
  - 1159pm Fri 10-10-2025
  - late submission receives no credit
- No class Mon 10-13-2025, Mon 10-20-2025
- Follow Monday schedule on: Tue 10-14-2025, Fri 10-24-2025
- Program 3 due date
  - 1159pm Fri 10-31-2025
  - late submission receives no credit
- Posted on Brightspace
  - Week 5 lecture slides

Queues

- A queue is a set of elements of the same type in which the elements are added at one end, called the back or rear, and deleted from the other end, called the front
- A queue of customers is seen in a bank or in a grocery store
- A queue of cars is seen waiting to pass through a tollbooth
- A queue of documents is seen waiting to be printed at a printer
- The customer at the front of a queue is served next
- When a new customer arrives, he or she stands at the end of the queue
- A queue is a First In First Out (FIFO) data structure

Queues

- Common queue operations:
  - initializeQueue - sets a queue to an empty state
  - isEmptyQueue - determines whether a queue is empty
  - isFullQueue - determines whether a queue is full
  - front - returns the first element of the a queue
  - addQueue - adds a new element to the rear of a queue
  - deleteQueue - removes the front element from a queue

Queues

- Standard template library (STL) queue object operations

TABLE 8-1 Operations on a `queue` object

Operation	Effect
`size`	Returns the actual number of elements in the queue.
`empty`	Returns **`true`** if the queue is empty, and **`false`** otherwise.
`push(item)`	Inserts a copy of `item` into the queue.
`front`	Returns the next—that is, first—element in the queue, but does not remove the element from the queue. This operation is implemented as a value-returning function.
`back`	Returns the last element in the queue, but does not remove the element from the queue. This operation is implemented as a value-returning function.
`pop`	Removes the next element in the queue.

Queues

- Sample code using STL queue operations

```
//*************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the STL class queue in a
// program.
//*************************************************************

#include <iostream>                                          //Line 1
#include <queue>                                             //Line 2

using namespace std;                                         //Line 3

int main()                                                   //Line 4
{                                                            //Line 5
    queue<int> intQueue;                                     //Line 6

    intQueue.push(26);                                       //Line 7
    intQueue.push(18);                                       //Line 8
    intQueue.push(50);                                       //Line 9
    intQueue.push(33);                                       //Line 10
```

Queues

- Sample code using STL queue operations

```
    cout << "Line 11: The front element of intQueue: "
         << intQueue.front() << endl;                    //Line 11

    cout << "Line 12: The last element of intQueue: "
         << intQueue.back() << endl;                     //Line 12

    intQueue.pop();                                      //Line 13

    cout << "Line 14: After the pop operation, the "
         << "front element of intQueue: "
         << intQueue.front() << endl;                    //Line 14

    cout << "Line 15: intQueue elements: ";              //Line 15

    while (!intQueue.empty())                            //Line 16
    {                                                    //Line 17
        cout << intQueue.front() << " ";                 //Line 18
        intQueue.pop();                                  //Line 19
    }                                                    //Line 20

    cout << endl;                                        //Line 21

    return 0;                                            //Line 22
}                                                        //Line 23
```

Graph definitions and notations

To facilitate and simplify our discussion, we borrow a few definitions and terminology from set theory. Let X be a set. If a is an element of X, we write $a \in X$. (The symbol "$\in$" means "belongs to.") A set Y is called a **subset** of X if every element of Y is also an element of X. If Y is a subset of X, we write $Y \subseteq X$. (The symbol "$\subseteq$" means "is a subset of.") The **intersection** of sets A and B, written $A \cap B$, is the set of all elements that are in A and B; that is, $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$. (The symbol "$\cap$" means "intersection.") The **union** of sets A and B, written $A \cup B$, is the set of all elements that are in A or in B; that is, $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$. (The symbol "$\cup$" means "union.") For sets A and B, the set $A \times B$ is the set of all ordered pairs of elements of A and B; that is, $A \times B = \{(a, b) \mid a \in A, b \in B\}$. (The symbol "$\times$" means "**Cartesian product**.")

Graph definitions and notations

A **graph** G is a pair, $G = (V, E)$, where V is a finite nonempty set, called the set of **vertices** of G and $E \subseteq V \times V$. That is, the elements of E are pairs of elements of V. E is called the set of **edges** of G. G is called **trivial** if it has only one vertex.

Let $V(G)$ denote the set of vertices, and $E(G)$ denote the set of edges of a graph G. If the elements of E are ordered pairs, G is called a **directed graph** or **digraph**; otherwise, G is called an **undirected graph**. In an undirected graph, the pairs (u, v) and (v, u) represent the same edge.

Binary tree definitions

Definition: A **binary tree**, T, is either empty or such that

i. T has a special node called the **root** node.

ii. T has two sets of nodes, L_T and R_T, called the **left subtree** and **right subtree** of T, respectively.

iii. L_T and R_T are binary trees.

Binary tree definitions

- Three traversals of a binary tree

Inorder Traversal

In an inorder traversal, the binary tree is traversed as follows:

1. Traverse the left subtree.
2. Visit the node.
3. Traverse the right subtree.

Preorder Traversal

In a preorder traversal, the binary tree is traversed as follows:

1. Visit the node.
2. Traverse the left subtree.
3. Traverse the right subtree.

Postorder Traversal

In a postorder traversal, the binary tree is traversed as follows:

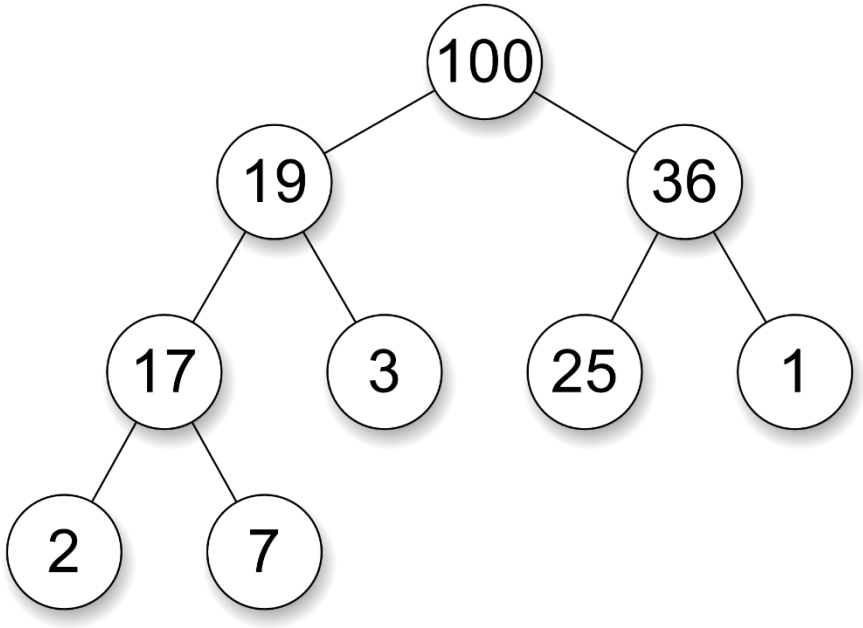
1. Traverse the left subtree.
2. Traverse the right subtree.
3. Visit the node.

Heaps

- In computer science, a **heap** is a tree-based data structure that satisfies the property:
  - if P is a parent node of C, then the key (the value) of P is either greater than or equal to (in a max heap) or less than or equal to (in a min heap) the key of C
- The node at the "top" of the heap (with no parents) is called the root node
- The heap is an efficient implementation of a data structure called a priority queue
- A common implementation of a heap is the binary heap, in which the tree is a binary tree
- Often, a binary heap is implemented in an array since indices of parent and child nodes are easily identified

Heaps

- Sample Max-heap



Array representation:

100	19	36	17	3	25	1	2	7
0	1	2	3	4	5	6	7	8

Assignment

- Read Malik chapter 8 - queues
- Read Malik chapter 10 – heaps
- Read Malik chapter 11 – binary trees
- Read Malik chapter 12 - graphs